

PENGEMBANGAN APLIKASI MOBILE

Event, State, Imperatif-Deklaratif, dan Implisit Intent



Dosen Pengampu:

Muhammad Aminul Akbar, S.Kom., M. T

Disusun oleh:

Najwa Tazkiya Rahman - 245150401111042

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS BRAWIJAYA

2026

Percobaan

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Pra-Percobaan

Pada percobaan ini dibuat dua project terpisah dengan hasil akhir yang sama. Teks awal adalah "Belum diklik". Setelah tombol "Ubah Teks" ditekan, teks berubah menjadi "Tombol sudah diklik". Perbedaan yang diamati bukan hasil akhirnya, melainkan cara program memperbarui UI.

Project A - XML/View (Imperatif)

1. Buat project Android Studio baru menggunakan template Empty Views Activity. Beri nama project ImperativeXmlApp dan pilih Kotlin.
2. Buka res/layout/activity_main.xml, kemudian gunakan kode berikut:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Belum diklik"
        android:textSize="24sp"
        android:layout_marginBottom="16dp" />

    <Button
        android:id="@+id/btnChange"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ubah Teks" />
</LinearLayout>
```

3. Buka MainActivity.kt. Sesuaikan package dengan package project, kemudian gunakan kode berikut:

```
package com.example.imperativexmlapp

import android.os.Bundle
```

```

import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

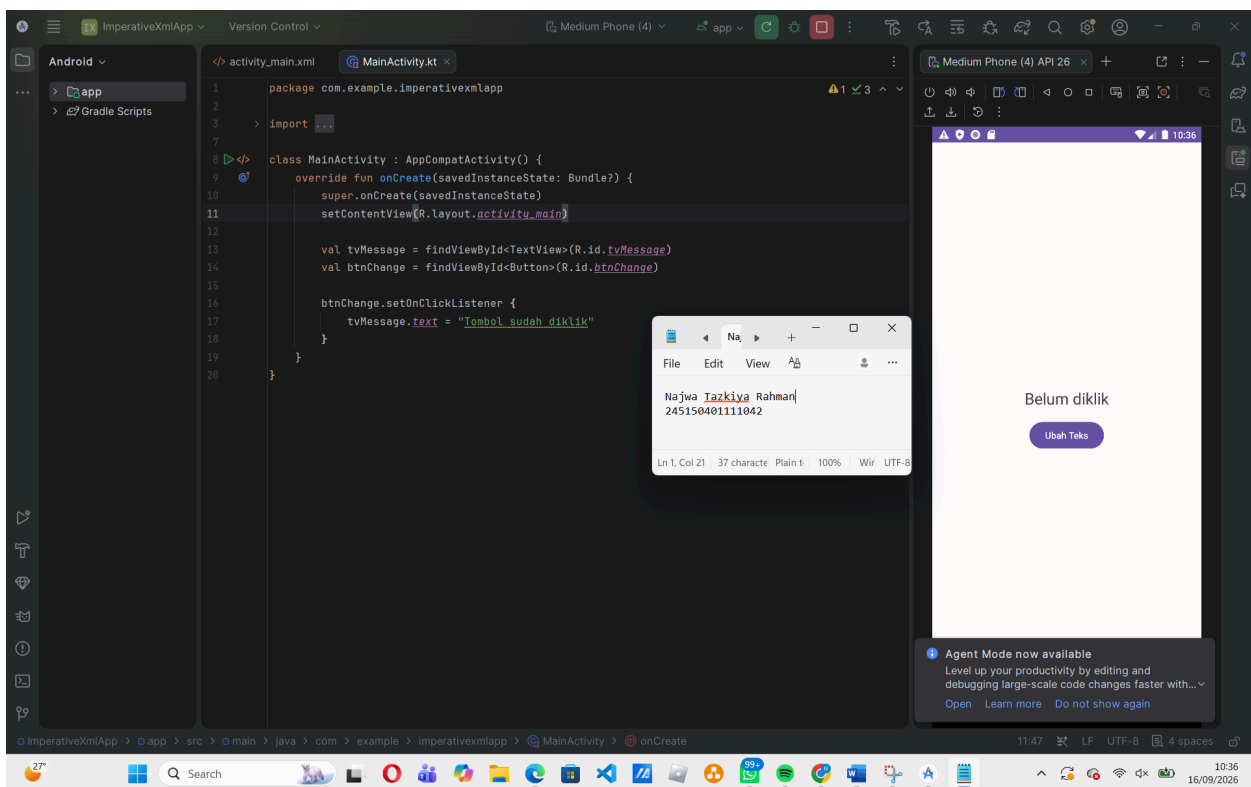
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

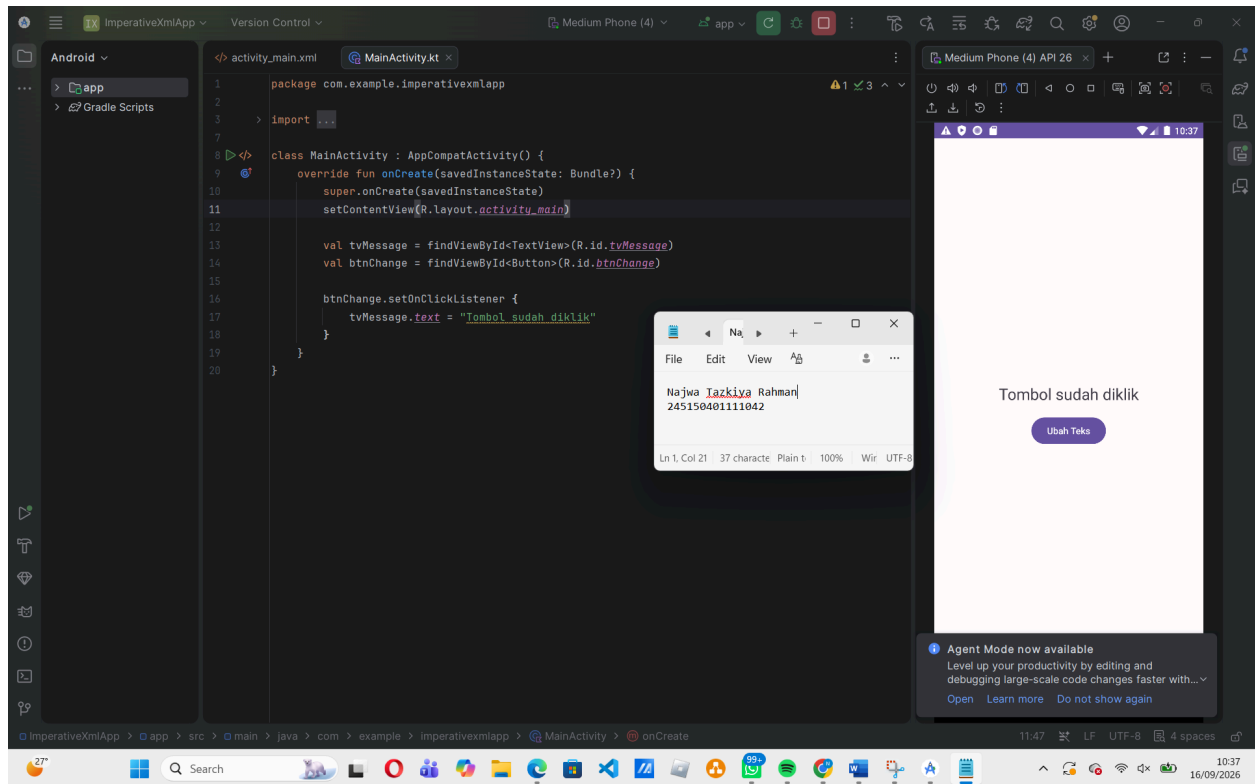
        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnChange = findViewById<Button>(R.id.btnChange)

        btnChange.setOnClickListener {
            tvMessage.text = "Tombol sudah diklik"
        }
    }
}

```

4. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.





Project B - Jetpack Compose (Deklaratif)

1. Buat project Android Studio baru menggunakan template Empty Activity untuk Jetpack Compose. Beri nama project DeclarativeComposeApp dan pilih Kotlin.
2. Buka MainActivity.kt, pertahankan package project, kemudian gunakan fungsi composable berikut:

@Composable

```
fun ChangeTextScreen() {
    var message by remember {
        mutableStateOf("Belum diklik")
    }
}
```

```
Column(
    modifier = Modifier.fillMaxSize(),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
) {
    Text(text = message, fontSize = 24.sp)
    Button(onClick = {
        message = "Tombol sudah diklik"
    }) {
        Text("Ubah Teks")
    }
}
```

```
}
```

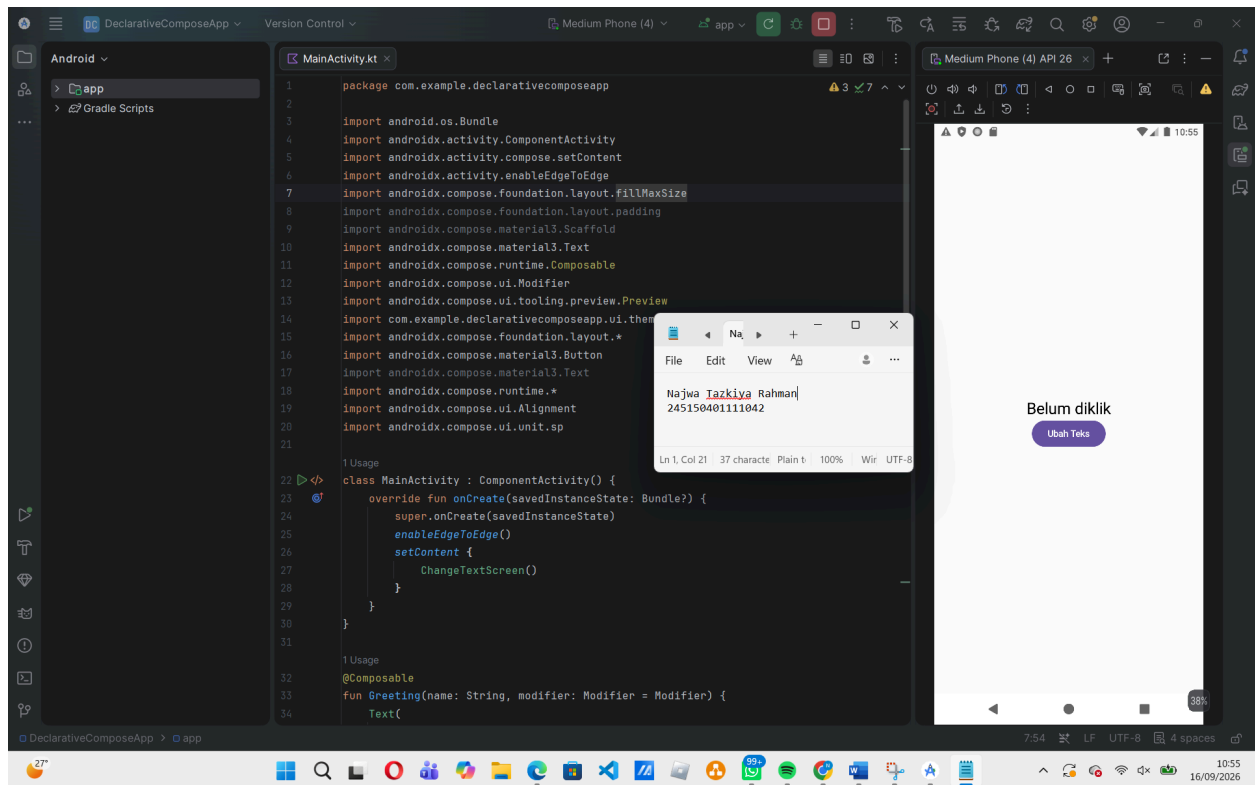
3. Tambahkan import yang diperlukan:

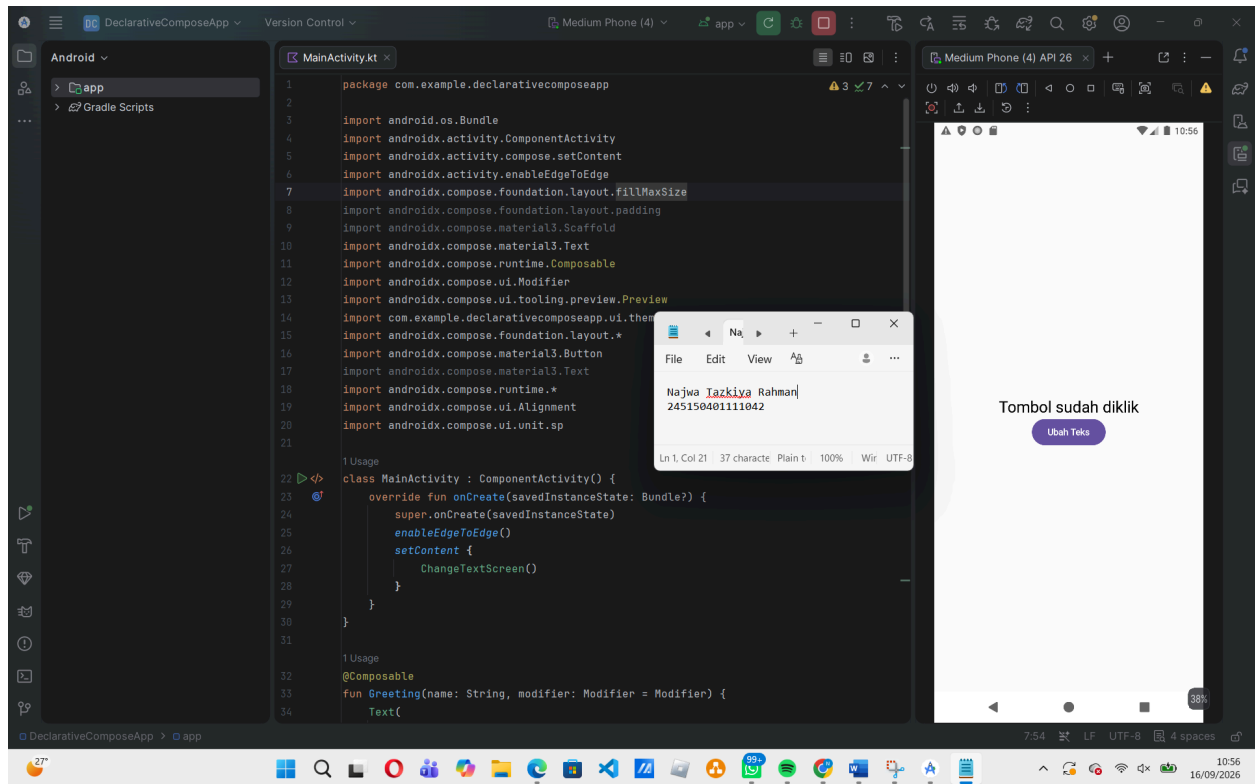
```
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.sp
```

4. Pada fungsi onCreate(), ubah isi setContent agar memanggil ChangeTextScreen():

```
setContent {
    ChangeTextScreen()
}
```

5. Jalankan aplikasi. Pastikan hasil tampilannya sama dengan project XML/View.





Aktivitas Percobaan

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

>> Objek yang diubah secara langsung adalah TextView dengan ID tvMessage. Baris kode yang melakukan perubahan adalah: `tvMessage.text = "Tombol sudah diklik"`. Pada program, tvMessage terlebih dahulu mengambil objek TextView melalui: `val tvMessage = findViewById<TextView>(R.id.tvMessage)`. Kemudian ketika tombol diklik, teks pada TextView diubah secara langsung menggunakan properti .text.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

>> Nilai yang diubah adalah state message. Deklarasi state terdapat pada:

```
var message by remember {
    mutableStateOf(value = "Belum diklik")
}
```

. Ketika tombol diklik, nilainya diubah melalui:

```
message = "Tombol sudah diklik"
```

. Jadi, berbeda dengan XML/View yang secara langsung mengubah TextView, Compose mengubah nilai state message.

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

>> Karena Jetpack Compose menggunakan pendekatan deklaratif. Program tidak perlu memberikan perintah secara langsung kepada objek UI untuk mengubah tampilannya. Pada Compose, cukup mengubah state yang menjadi sumber data tampilan. Ketika state berubah, Compose akan melakukan recomposition dan memperbarui bagian UI yang bergantung pada state tersebut.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

>> Ketika menekan tombol "Ubah Teks", terjadi event `onClick`. Event tersebut kemudian menjalankan perintah: `tvMessage.text = "Tombol sudah diklik"`. Perintah tersebut secara langsung mengubah objek `TextView`, sehingga teks yang ditampilkan pada layar berubah dari "Belum diklik" menjadi "Tombol sudah diklik".

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

>> Ketika menekan tombol "Ubah Teks", event `onClick` dijalankan. Event tersebut mengubah state: `message = "Tombol sudah diklik"`. Karena state berubah, Compose mendeteksi perubahan tersebut dan melakukan recomposition terhadap bagian UI yang bergantung pada state `message`. Kemudian `Text(text = message, fontSize = 24.sp)` akan menampilkan nilai terbaru, yaitu "Tombol sudah diklik".

7. Apa yang terjadi jika `message` diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

```
var message by remember {  
    mutableStateOf(value = "Belum diklik")  
}
```

>> Jika kode `message = "Tombol sudah diklik"` diganti menjadi `message = "Tombol sudah diklik"` dijalankan, perubahan tersebut tidak menyebabkan Compose melakukan recomposition sebagaimana ketika menggunakan `mutableStateOf`. Akibatnya, teks pada UI tidak diperbarui secara otomatis.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

>> Pendekatan imperatif berfokus pada perintah atau langkah yang harus dilakukan untuk mengubah tampilan. Contohnya, ketika tombol diklik, program secara langsung memberikan

perintah kepada TextView untuk mengubah teks. Sedangkan pendekatan deklaratif berfokus pada kondisi UI yang ingin ditampilkan berdasarkan state. Programmer hanya cukup mengubah state, kemudian Compose akan mengatur pembaruan tampilan melalui recomposition. Jadi, imperatif lebih berfokus bagaimana UI diubah, sedangkan deklaratif lebih berfokus pada UI seperti apa yang harus ditampilkan berdasarkan state.

Event Handling State

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity:

@Composable

```
fun EventStateTest() {  
    var count = 0  
    Column(  
        verticalArrangement = Arrangement.Center,  
        horizontalAlignment = Alignment.CenterHorizontally,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        Text("Counter : $count")  
        Button(onClick = {  
            count++  
            Log.d("TEST", "Counter berubah : $count")  
        }) {  
            Text("Tambah")  
        }  
    }  
}
```

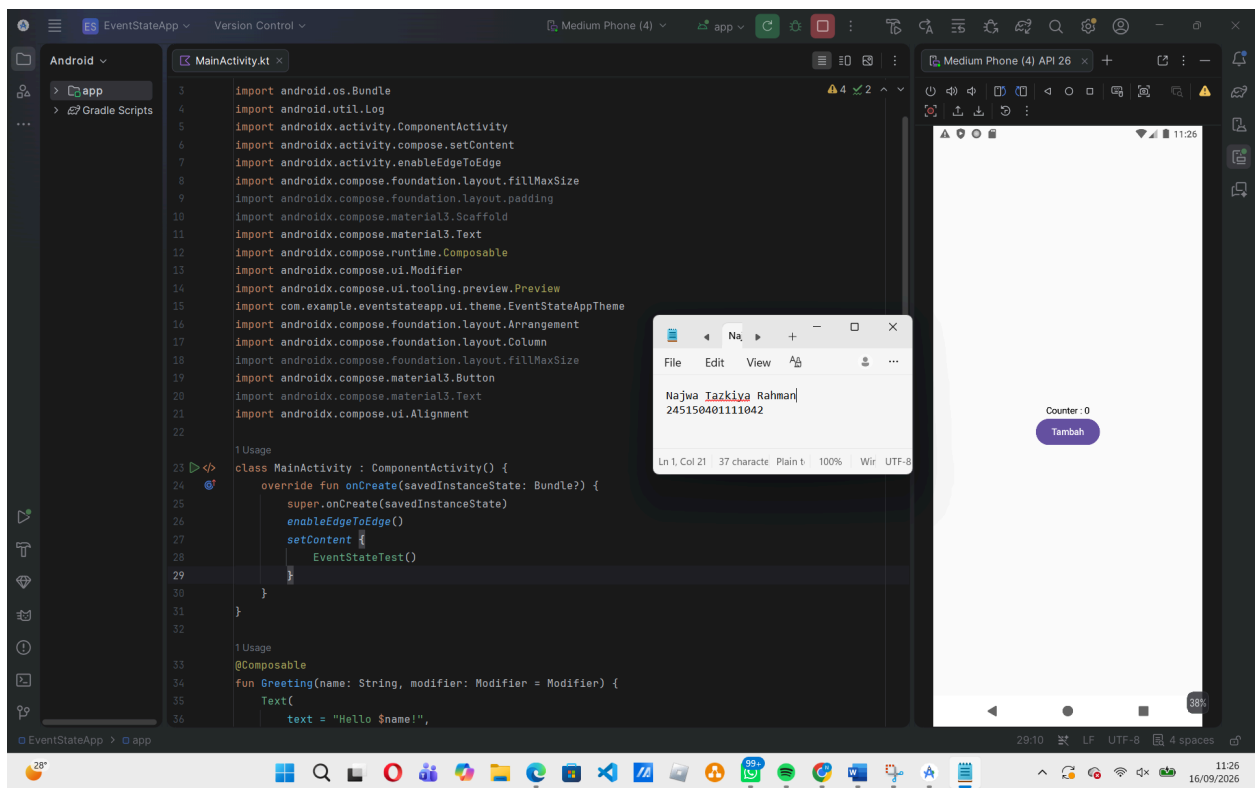
3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

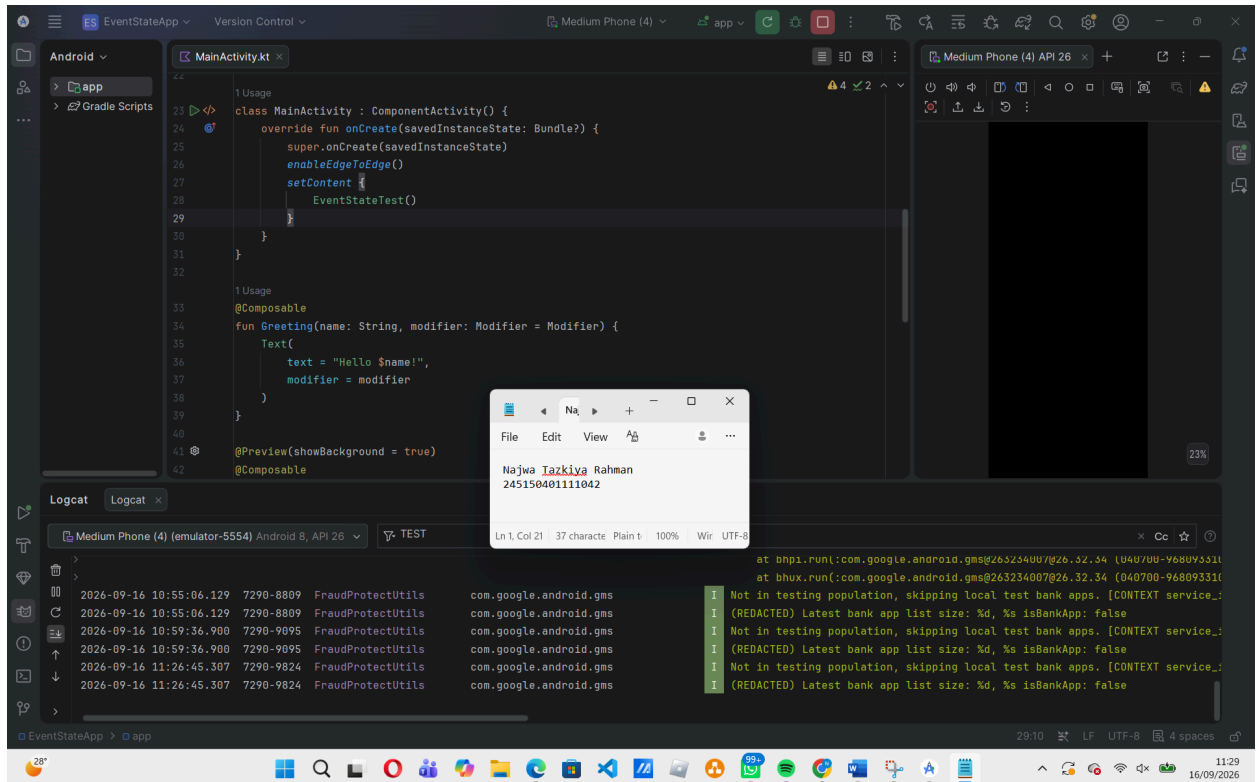
```
setContent {
    EventStateTest()
}
```

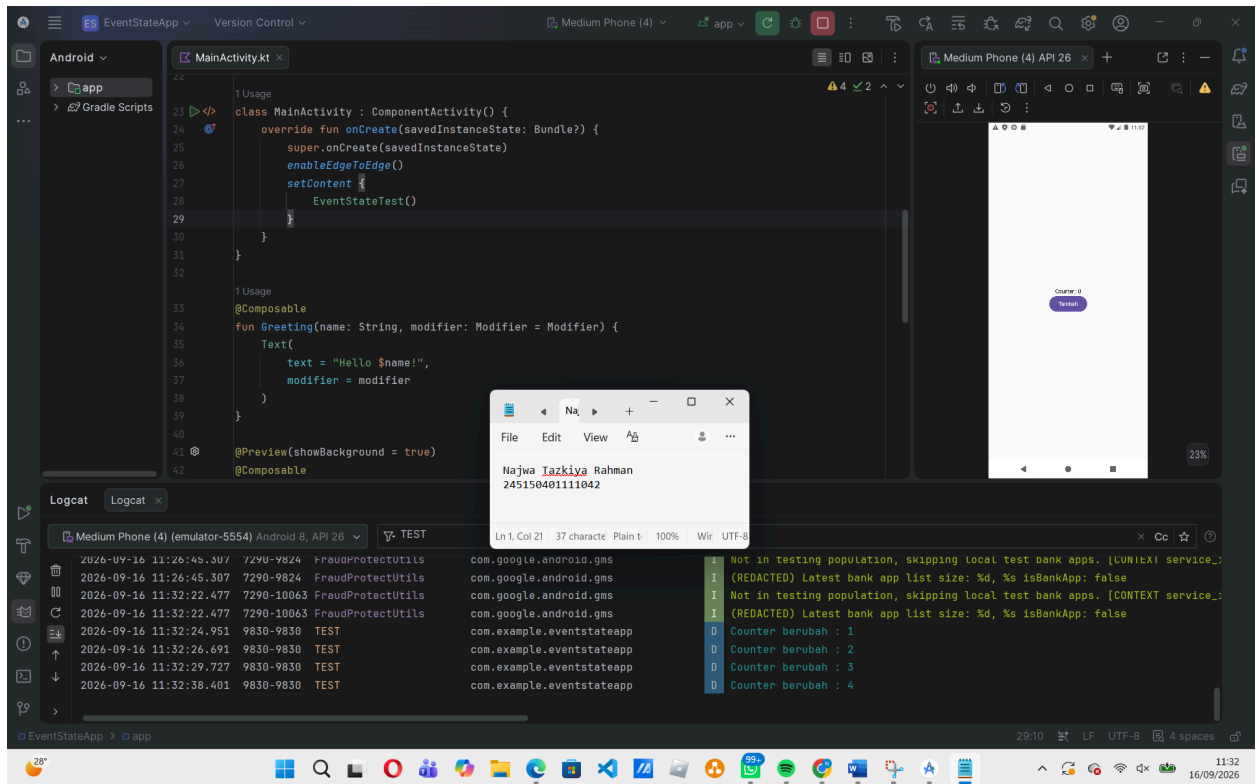
5. Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.



Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

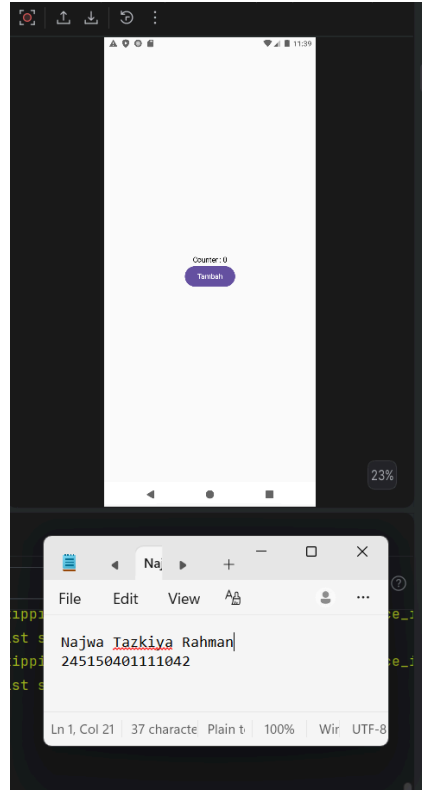




2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

2026-09-16 11:32:38.401 9830-9830 TEST com.example.eventstateapp D Counter berubah : 4

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?



4. Apa yang menyebabkan hal tersebut terjadi?

>> Hal tersebut terjadi karena count merupakan variabel biasa, bukan state Compose. Compose tidak menganggap perubahan pada: `var count = 0` sebagai perubahan state yang harus menyebabkan recomposition. Akibatnya, meskipun `count++` dijalankan, Compose tidak mendapatkan perubahan state yang dapat digunakan untuk memperbaiki UI.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable `EventStateTest`.

6. Ubah kode program pada bagian fungsi Composable `EventStateTest` dari yang sebelumnya:

```
var count = 0
```

menjadi:

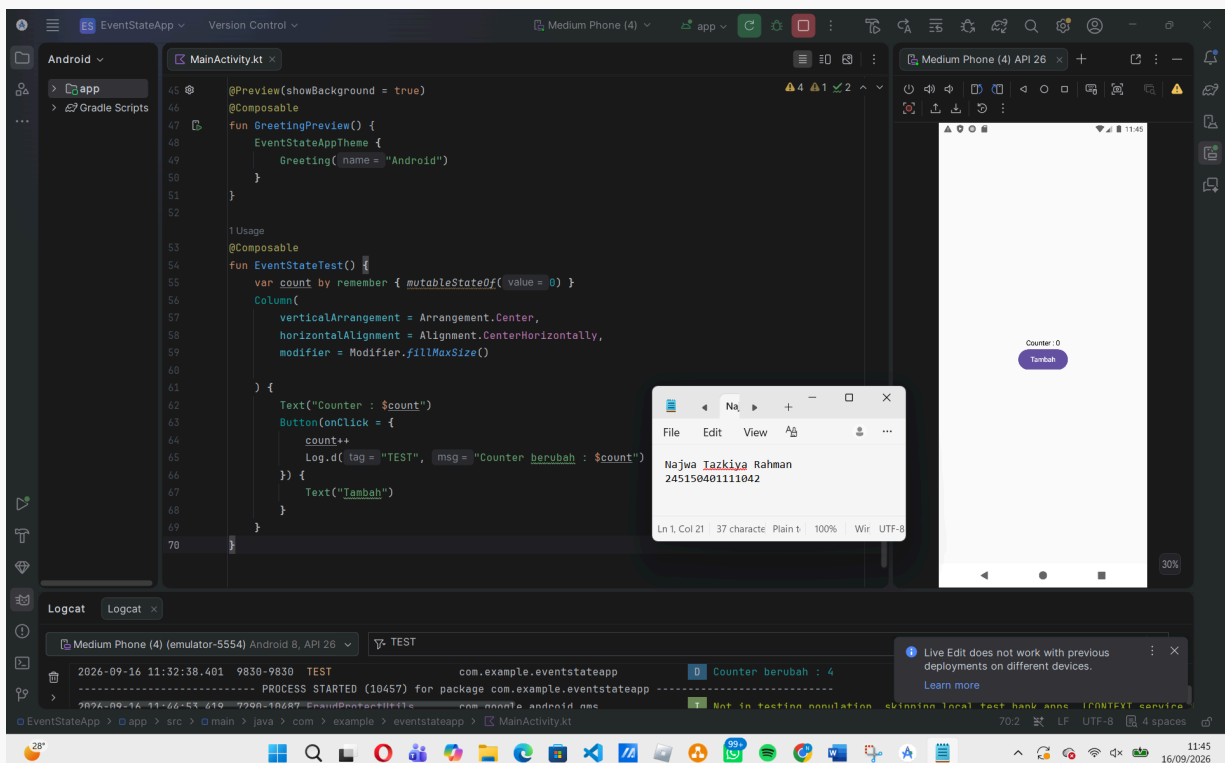
```
var count by remember { mutableStateOf(0) }
```

7. Berikut kode lengkap `EventStateTest`:

```

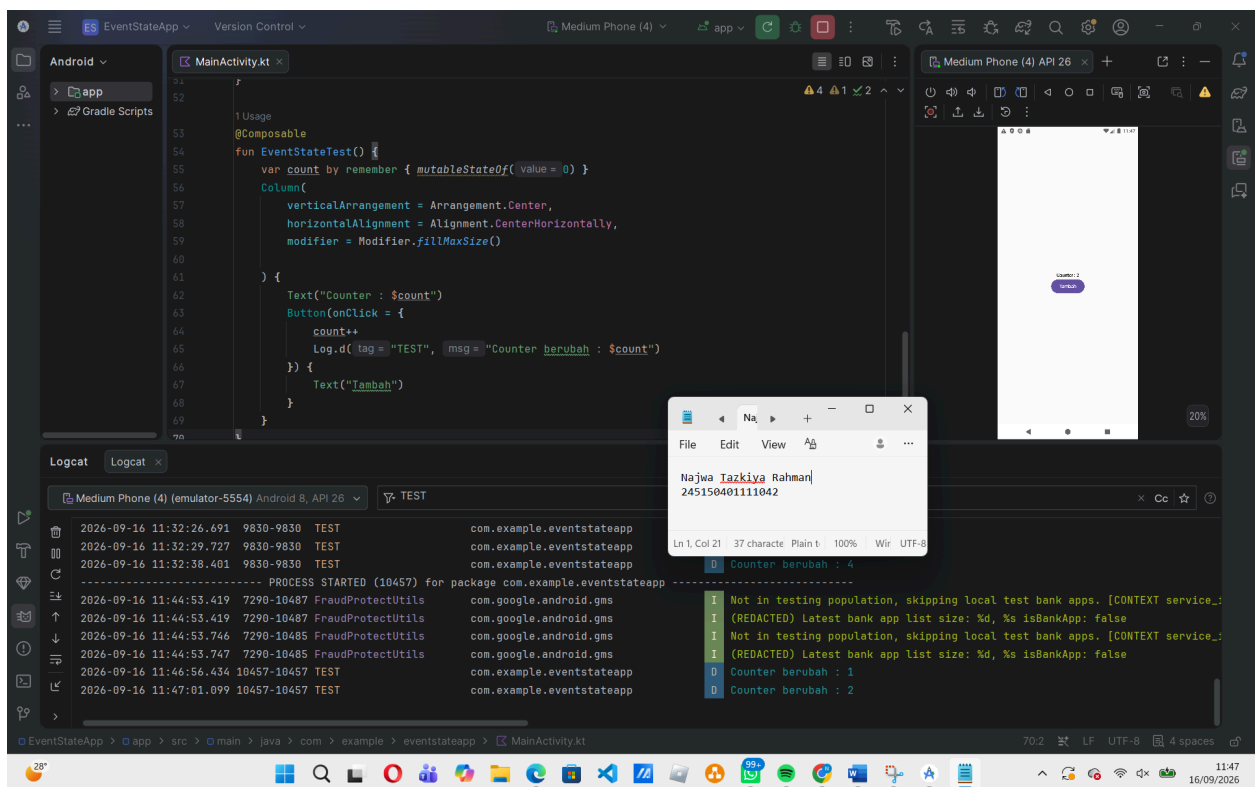
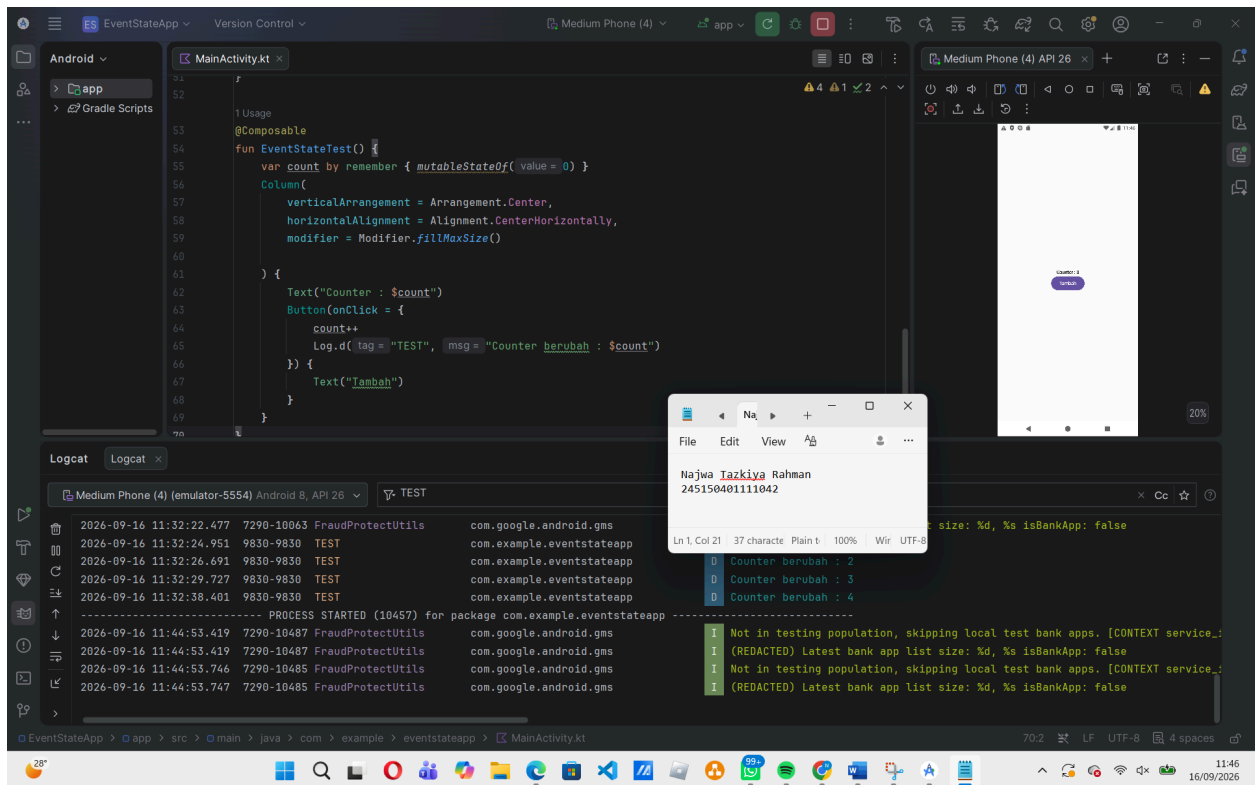
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}

```



- Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan

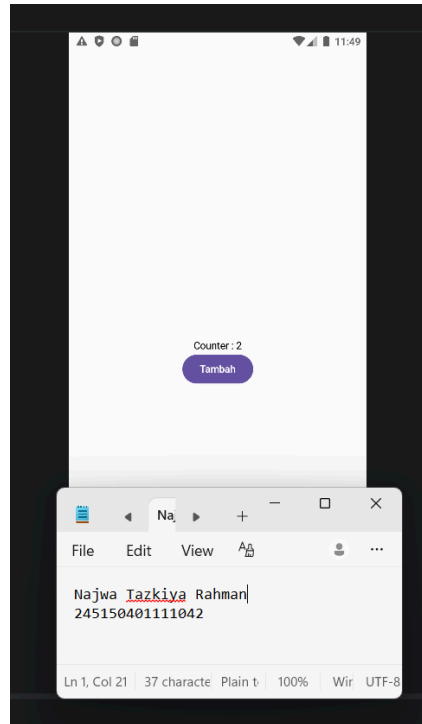
TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

```
2026-09-16 11:46:56.434 10457-10457 TEST com.example.eventstateapp D Counter berubah : 1
2026-09-16 11:47:01.099 10457-10457 TEST com.example.eventstateapp D Counter berubah : 2
```

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?



11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

>> Perbedaannya disebabkan oleh penggunaan state. Pada percobaan sebelumnya, count merupakan variabel biasa sehingga perubahan nilainya tidak menyebabkan Compose memperbarui UI. Pada percobaan kedua, count menggunakan: `remember { mutableStateOf(0) }` sehingga ketika `count++` dijalankan, state berubah. Perubahan state tersebut menyebabkan Compose melakukan recomposition sehingga UI menampilkan nilai count terbaru.

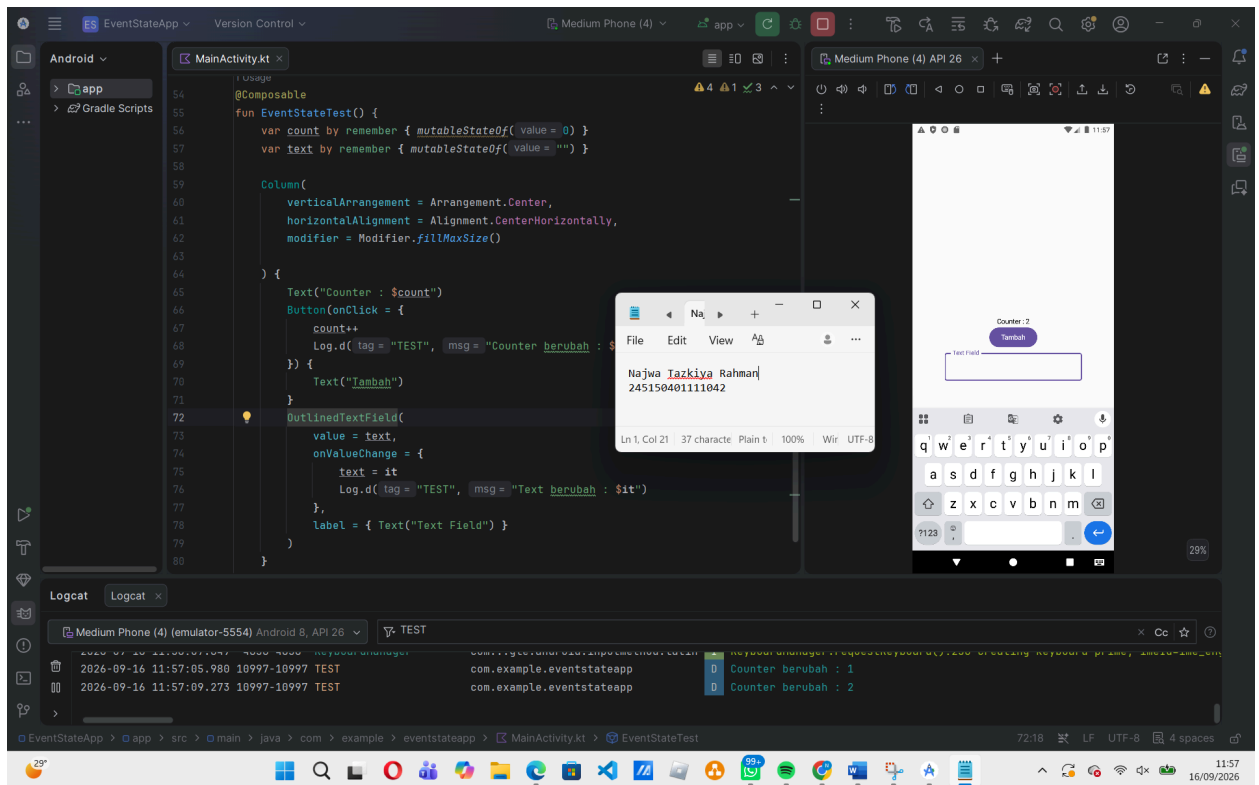
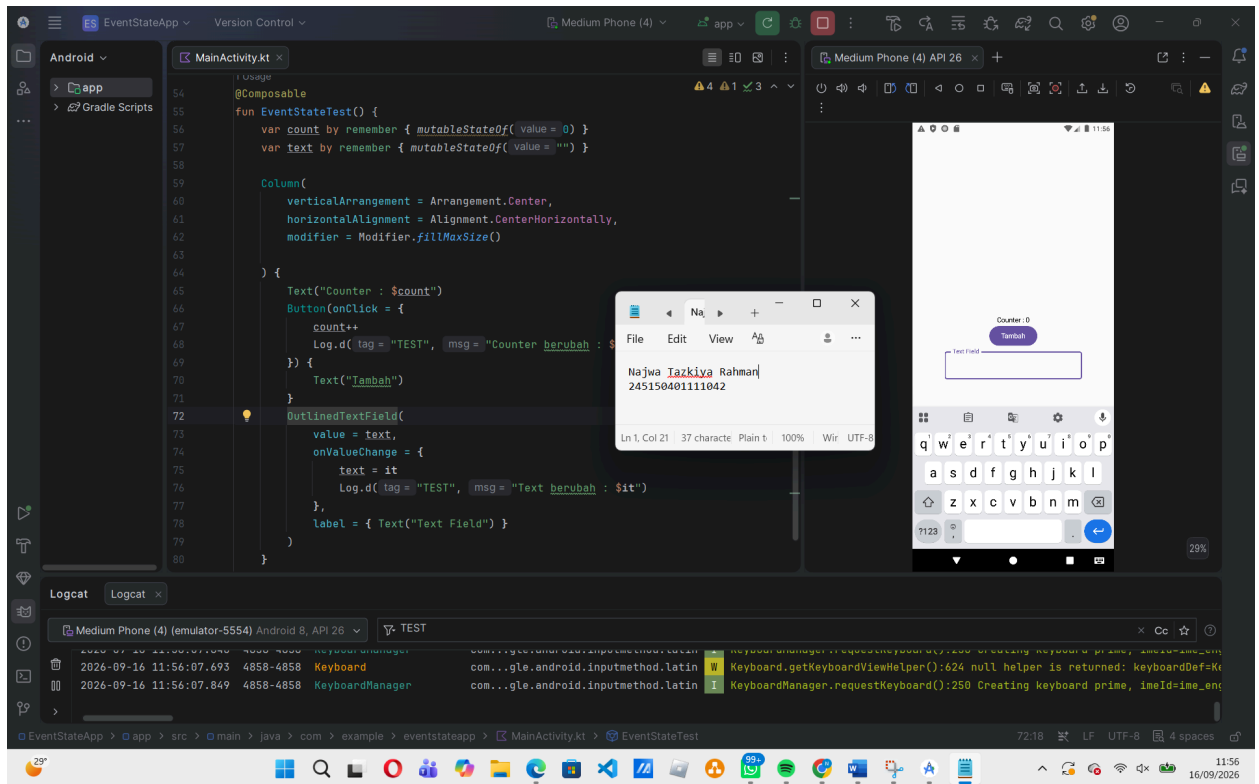
12. Apa yang dimaksud recomposition pada Compose?

>> Recomposition adalah proses ketika Compose menjalankan kembali bagian composable yang bergantung pada state yang mengalami perubahan, sehingga UI dapat diperbarui sesuai dengan nilai state terbaru.

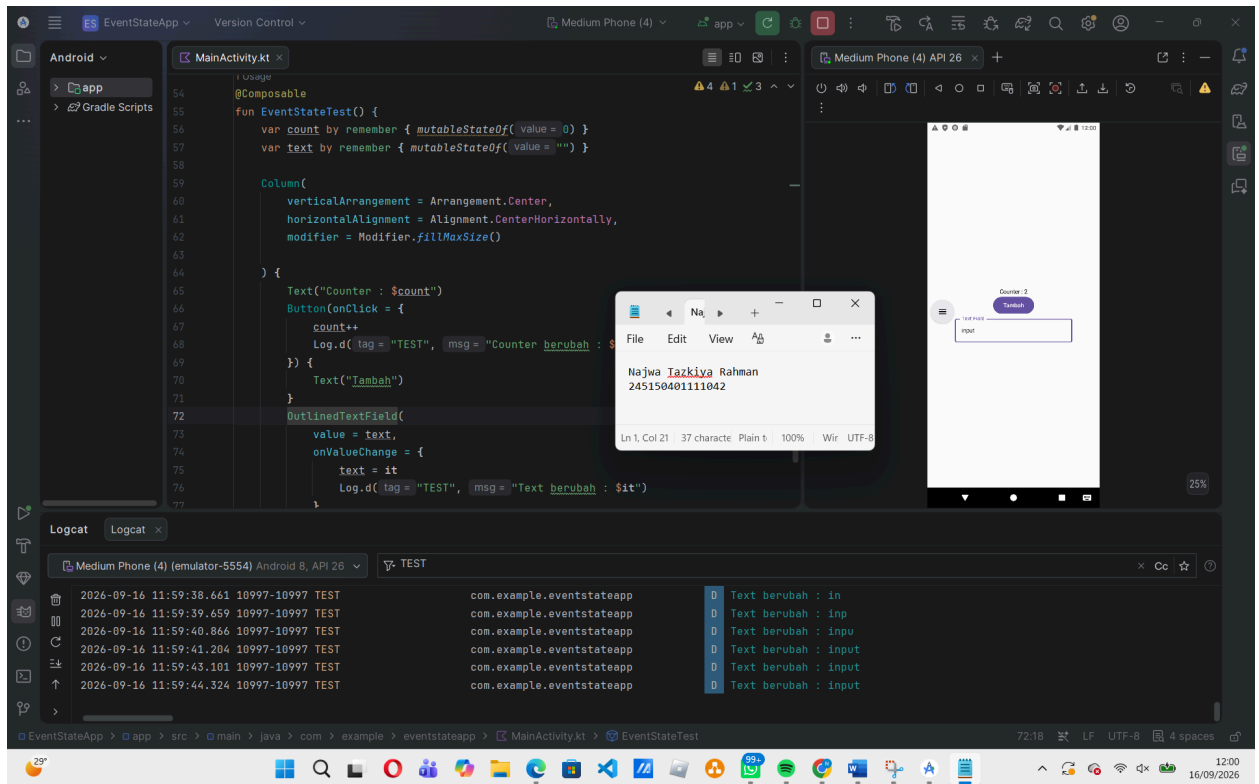
13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

```
fun EventStateTest() {  
    var count by remember { mutableStateOf(0) }  
    var text by remember { mutableStateOf("") }  
  
    Column(  
        verticalArrangement = Arrangement.Center,  
        horizontalAlignment = Alignment.CenterHorizontally,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        Text("Counter : $count")  
        Button(onClick = {  
            count++  
            Log.d("TEST", "Counter berubah : $count")  
        }) {  
            Text("Tambah")  
        }  
        OutlinedTextField(  
            value = text,  
            onValueChange = {  
                text = it  
                Log.d("TEST", "Text berubah : $it")  
            },  
            label = { Text("Text Field") }  
        )  
    }  
}
```

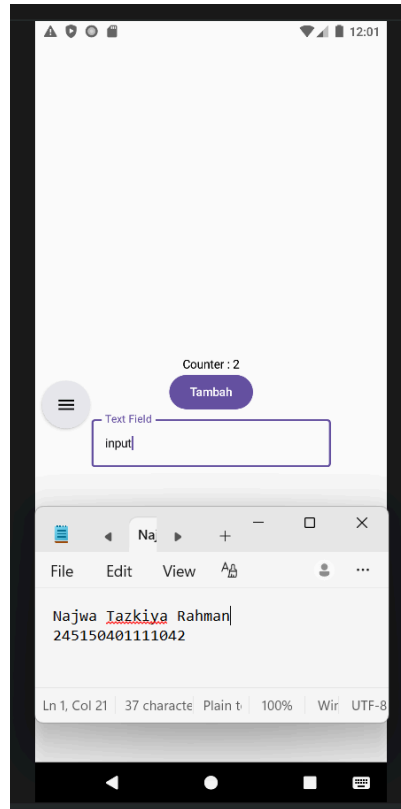
14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?



16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?



17. Apa yang menyebabkan hal tersebut terjadi?

>> Hal tersebut terjadi karena setiap perubahan input pada Text Field memicu event

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

>> Terdapat dua state, yaitu:

- count

var count by remember { mutableStateOf(0) }-> Digunakan untuk menyimpan jumlah klik tombol Tambah.

- text

var text by remember { mutableStateOf("") }-> Digunakan untuk menyimpan teks yang dimasukkan pada OutlinedTextField.

Kedua state tersebut terdapat pada program terakhir dalam percobaan Event Handling dan State.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

>> Ada 2 event utama:

- Event klik tombol Tambah ->

```
Button(onClick = {
    count++
})
```

- Event perubahan nilai Text Field ->

```
onValueChange = {
    text = it
}
```

Keduanya merupakan event yang menyebabkan perubahan pada state.

20. Kapan event - event tersebut dijalankan?

Event `onClick` dijalankan ketika pengguna menekan tombol Tambah. Event `onValueChange` dijalankan ketika pengguna memasukkan atau mengubah karakter pada Text Field.

Implisit Intent

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file `MainActivity.kt` dan tambahkan fungsi composable berikut ini di luar class `MainActivity`

`@Composable`

```
fun ImplicitIntentTest(context: Context) {
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        // Buka Browser
        Button(onClick = {
            val intent = Intent(Intent.ACTION_VIEW).apply {
                data = Uri.parse("https://www.google.com")
            }
            context.startActivity(intent)
        }) {
            Text("Buka Browser")
        }
        Spacer(modifier = Modifier.height(16.dp))
        // Buka Dial Telepon
    }
}
```

```

Button(onClick = {
    val intent = Intent(Intent.ACTION_DIAL).apply {
        data = Uri.parse("tel:08123456789")
    }
    context.startActivity(intent)
}) {
    Text("Buka Telepon")
}
Spacer(modifier = Modifier.height(16.dp))
// Kirim Email (dengan chooser)
Button(onClick = {
    val intent = Intent(Intent.ACTION_SENDTO).apply {
        data = Uri.parse("mailto:test@gmail.com")
    }
    val chooser = Intent.createChooser(intent, "Pilih aplikasi email")
    context.startActivity(chooser)
}) {
    Text("Kirim Email")
}
Spacer(modifier = Modifier.height(16.dp))
// Buka Maps
Button(onClick = {
    val intent = Intent(Intent.ACTION_VIEW).apply {
        data = Uri.parse("geo:-7.250445,112.768845")
    }
    context.startActivity(intent)
}) {
    Text("Buka Maps")
}
}
}

```

3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```

import android.content.Context
import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button

```

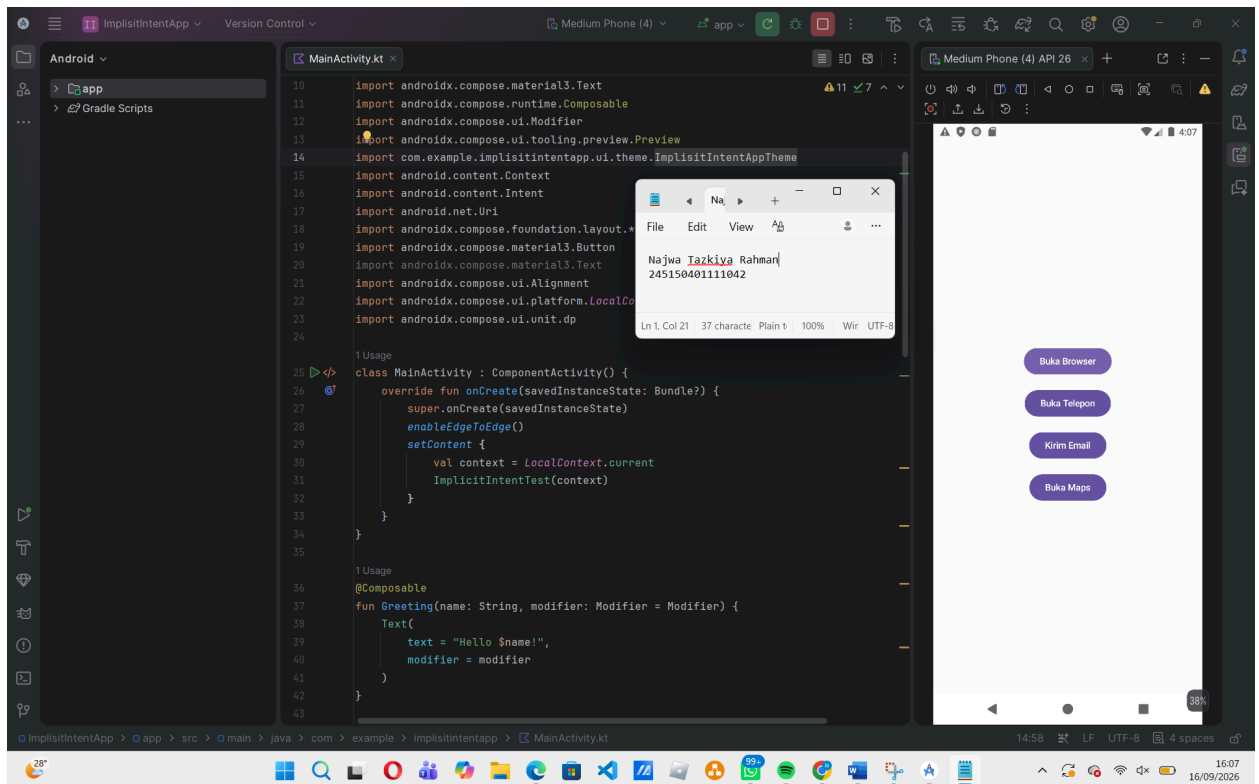
```
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

```
setContent {
    val context = LocalContext.current
    ImplicitIntentTest(context)
}
```

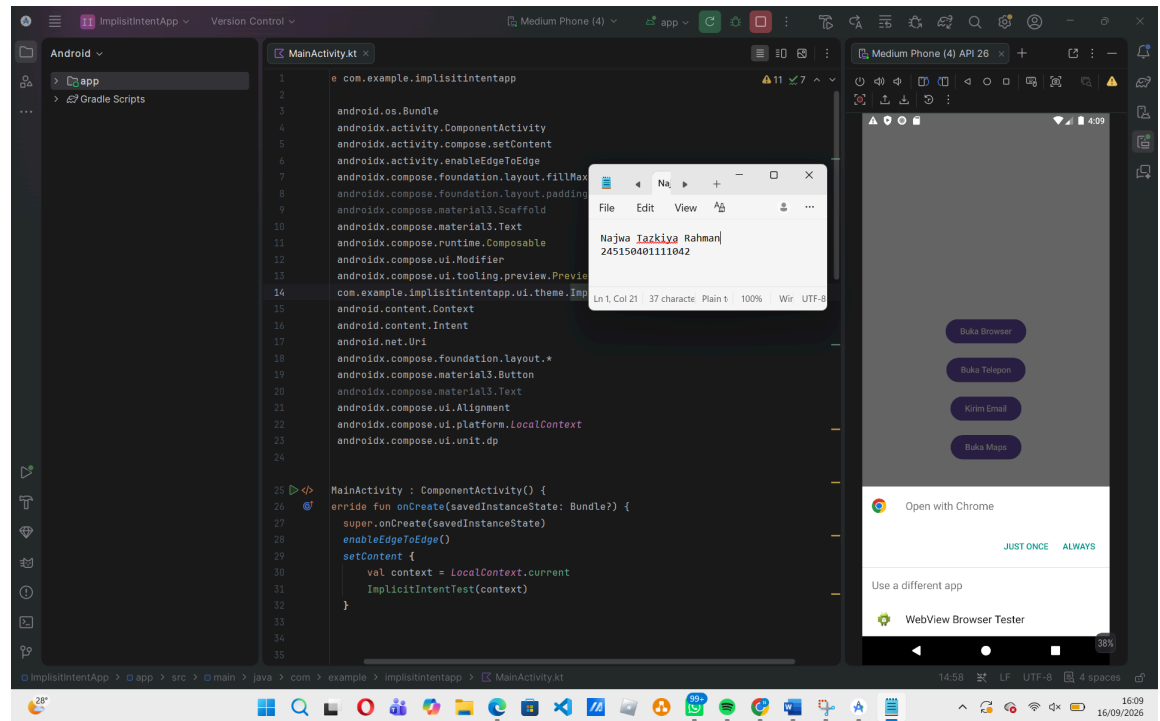
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

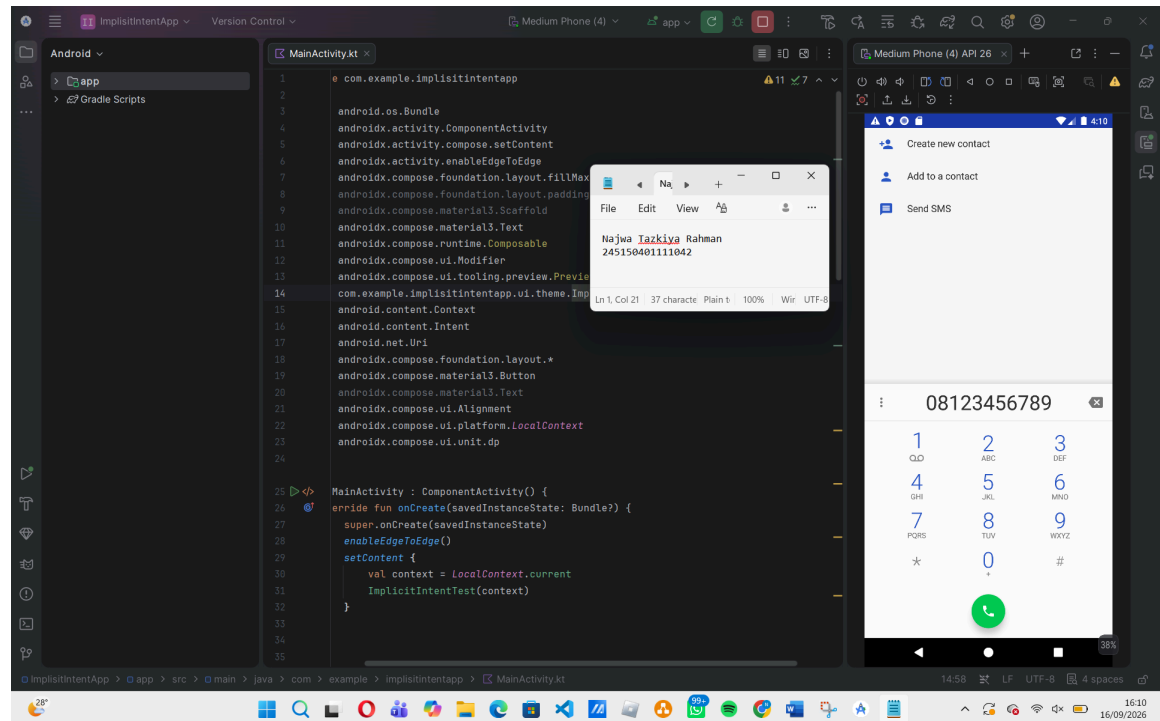


2. Klik Buka Browser
3. Klik Buka Telepon
4. Klik Kirim Email
5. Klik Buka Maps

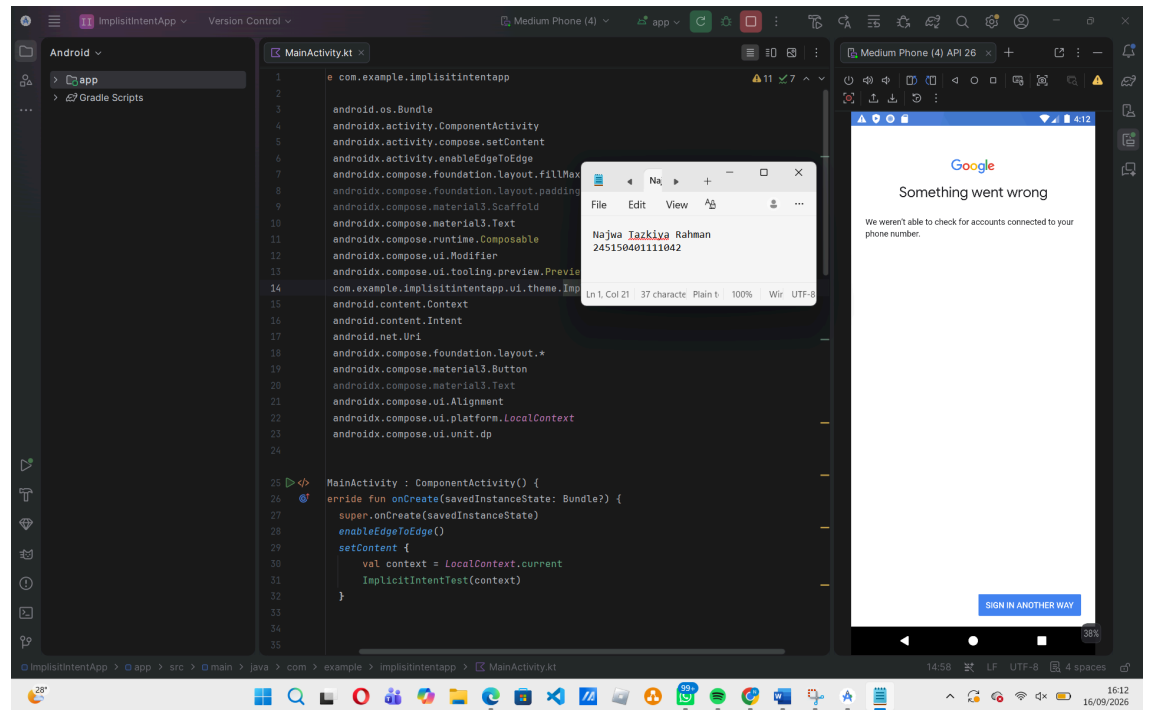
6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya
- Buka Browser



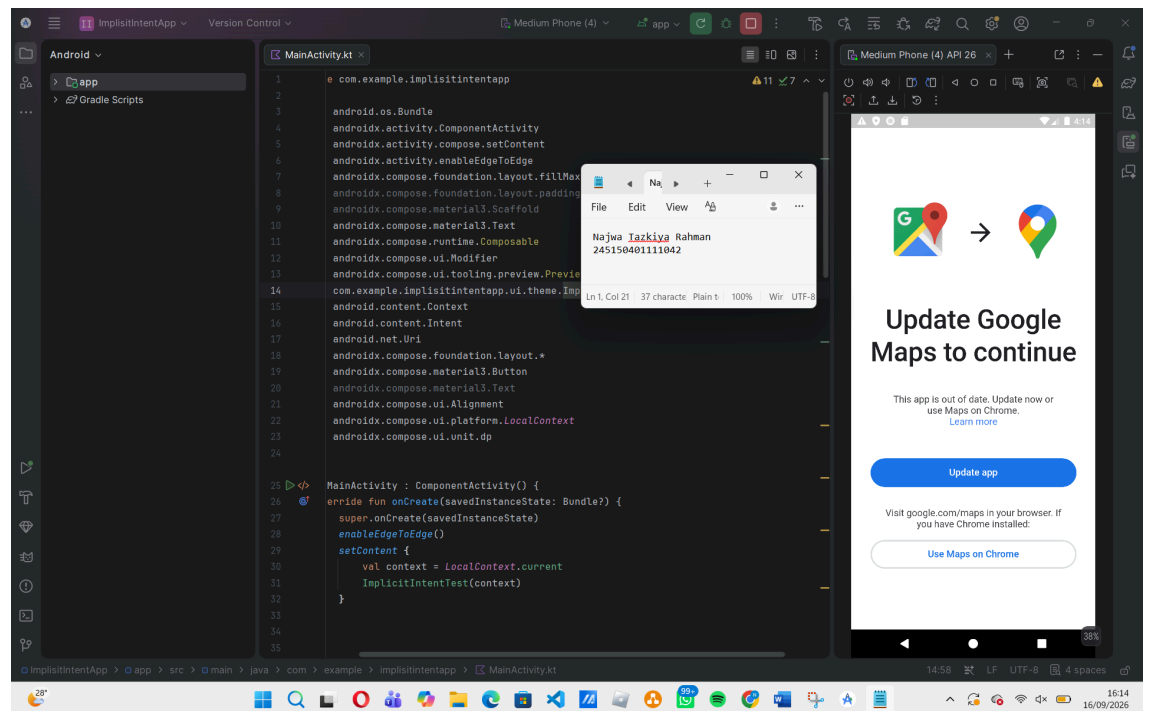
- Buka Telepon



- Kirim Email



- Buka Maps



7. Pada kode yang digunakan, terdapat beberapa action seperti:
8. Intent.ACTION_VIEW
9. Intent.ACTION_DIAL
10. Intent.ACTION_SENDTO

11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

- Intent.ACTION_VIEW -> Digunakan untuk menampilkan atau membuka suatu data menggunakan aplikasi yang sesuai.
- Intent.ACTION_DIAL -> Digunakan untuk membuka aplikasi dialer/telepon dengan nomor tertentu.
- Intent.ACTION_SENDTO -> Digunakan untuk melakukan aksi pengiriman kepada tujuan tertentu berdasarkan URI yang diberikan.

12. Pada kode yang digunakan, terdapat beberapa data seperti:

13. data = Uri.parse("mailto:test@gmail.com")

14. data = Uri.parse("<https://www.google.com>")

15. data = Uri.parse("geo:-7.250445,112.768845")

16. Apa fungsi data pada intent ?

>> Data berfungsi untuk memberikan informasi atau URI yang menjadi target dari action Intent. Contohnya: <https://www.google.com> merupakan alamat website yang ingin dibuka.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

>> Action dan Data bekerja bersama untuk menentukan aplikasi yang sesuai untuk menangani sebuah Intent. Action menunjukkan jenis tindakan yang ingin dilakukan, sedangkan Data menunjukkan data atau target yang digunakan dalam tindakan tersebut. Keduanya digunakan oleh sistem Android untuk menentukan aplikasi mana yang dapat menangani Intent tersebut.